

Competitive Programming

Section 36 Editorial: Fenwick Trees

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	4
2	A. Point Add Range Sum	5
2.1	Restatement	5
2.2	Algorithm	5
2.3	Correctness proof	5
2.4	Complexity	5
2.5	Full C++ solution	5
2.6	Full Python solution	6
3	B. Inversion Count	8
3.1	Restatement	8
3.2	Algorithm	8
3.3	Correctness proof	8
3.4	Complexity	8
3.5	Full C++ solution	8
3.6	Full Python solution	9
4	C. Range Add Point Query	11
4.1	Restatement	11
4.2	Algorithm	11
4.3	Correctness proof	11
4.4	Complexity	11
4.5	Full C++ solution	11
4.6	Full Python solution	12
5	D. Range Add Range Sum	14
5.1	Restatement	14
5.2	Constraints and traps	14
5.3	Observations	14
5.4	Sample walk-through	15
5.5	Algorithm	15
5.6	Correctness proof	16
5.7	Complexity	16
5.8	C++ implementation notes	16
5.9	Python implementation notes	16
5.10	Full C++ solution	17
5.11	Full Python solution	18
5.12	Common wrong answers	19
5.13	Tests that matter	19
6	Testing discussion	20
7	Problem-specific upsolve notes	21
7.1	A. Point Add Range Sum	21

7.2	B. Inversion Count	21
7.3	C. Range Add Point Query	22
8	Stress-testing workflow	23

1 Editorial overview

Problem	Main idea
A. Point Add Range Sum	See full algorithm and code below.
B. Inversion Count	See full algorithm and code below.
C. Range Add Point Query	See full algorithm and code below.
D. Range Add Range Sum	Difference array plus two weighted prefix trees.

2 A. Point Add Range Sum

2.1 Restatement

Maintain an array. Operation 1 $i\ x$ adds x to $a[i]$. Operation 2 $l\ r$ asks for the sum of $a[l..r]$.

2.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

2.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

2.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<long long> bit;
```

```

Fenwick(int n) : n(n), bit(n + 1, 0) {}
void add(int i, long long delta) {
    for (; i <= n; i += i & -i) bit[i] += delta;
}
long long sum(int i) const {
    long long res = 0;
    for (; i > 0; i -= i & -i) res += bit[i];
    return res;
}
long long range_sum(int l, int r) const {
    return sum(r) - sum(l - 1);
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    Fenwick bit(n);
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        bit.add(i, x);
    }
    while (q--) {
        int typ;
        cin >> typ;
        if (typ == 1) {
            int i;
            long long x;
            cin >> i >> x;
            bit.add(i, x);
        } else {
            int l, r;
            cin >> l >> r;
            cout << bit.range_sum(l, r) << '\n';
        }
    }
    return 0;
}

```

2.6 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n):

```

```

        self.n = n
        self.bit = [0] * (n + 1)
    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
    def sum(self, i):
        res = 0
        while i > 0:
            res += self.bit[i]
            i -= i & -i
        return res
    def range_sum(self, l, r):
        return self.sum(r) - self.sum(l - 1)

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    bit = Fenwick(n)
    for i, x in enumerate(a, 1):
        bit.add(i, x)
    idx = 2 + n
    out = []
    for _ in range(q):
        typ = data[idx]
        idx += 1
        if typ == 1:
            i, x = data[idx], data[idx + 1]
            idx += 2
            bit.add(i, x)
        else:
            l, r = data[idx], data[idx + 1]
            idx += 2
            out.append(str(bit.range_sum(l, r)))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

3 B. Inversion Count

3.1 Restatement

Given an array, count pairs $i < j$ with $a[i] > a[j]$. Values may repeat and may be large.

3.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

3.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

3.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;
    Fenwick(int n) : n(n), bit(n + 1, 0) {}
};
```

```

void add(int i, int delta) {
    for (; i <= n; i += i & -i) bit[i] += delta;
}

int sum(int i) const {
    int res = 0;
    for (; i > 0; i -= i & -i) res += bit[i];
    return res;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n), vals;
    for (long long &x : a) {
        cin >> x;
        vals.push_back(x);
    }
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    Fenwick bit((int)vals.size());
    long long inv = 0;
    for (int i = 0; i < n; i++) {
        int rank = int(lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin()) + 1;
        inv += i - bit.sum(rank);
        bit.add(rank, 1);
    }
    cout << inv << '\n';
    return 0;
}

```

3.6 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)
    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
    def sum(self, i):
        res = 0
        while i > 0:

```

```

        res += self.bit[i]
        i -= i & -i
    return res

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    a = data[1:1 + n]
    values = {x: i + 1 for i, x in enumerate(sorted(set(a)))}
    bit = Fenwick(len(values))
    inv = 0
    seen = 0
    for x in a:
        rank = values[x]
        inv += seen - bit.sum(rank)
        bit.add(rank, 1)
        seen += 1
    print(inv)

if __name__ == "__main__":
    main()

```

4 C. Range Add Point Query

4.1 Restatement

Maintain an array. Operation 1 $l\ r\ x$ adds x to every element in $[l, r]$. Operation 2 i asks for the current value of $a[i]$.

4.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

4.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

4.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<long long> bit;
```

```

Fenwick(int n) : n(n), bit(n + 2, 0) {}
void add(int i, long long delta) {
    for (; i <= n; i += i & -i) bit[i] += delta;
}
long long sum(int i) const {
    long long res = 0;
    for (; i > 0; i -= i & -i) res += bit[i];
    return res;
}
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    Fenwick bit(n + 1);
    long long prev = 0;
    for (int i = 1; i <= n; i++) {
        long long x;
        cin >> x;
        bit.add(i, x - prev);
        prev = x;
    }
    while (q--) {
        int typ;
        cin >> typ;
        if (typ == 1) {
            int l, r;
            long long x;
            cin >> l >> r >> x;
            bit.add(l, x);
            bit.add(r + 1, -x);
        } else {
            int i;
            cin >> i;
            cout << bit.sum(i) << '\n';
        }
    }
    return 0;
}

```

4.6 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n):

```

```

        self.n = n
        self.bit = [0] * (n + 2)
    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
    def sum(self, i):
        res = 0
        while i > 0:
            res += self.bit[i]
            i -= i & -i
        return res

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    bit = Fenwick(n + 1)
    prev = 0
    for i, x in enumerate(a, 1):
        bit.add(i, x - prev)
        prev = x
    idx = 2 + n
    out = []
    for _ in range(q):
        typ = data[idx]
        idx += 1
        if typ == 1:
            l, r, x = data[idx], data[idx + 1], data[idx + 2]
            idx += 3
            bit.add(l, x)
            bit.add(r + 1, -x)
        else:
            i = data[idx]
            idx += 1
            out.append(str(bit.sum(i)))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

5 D. Range Add Range Sum

5.1 Restatement

An initially zero array receives interval additions. Between updates, queries ask for the sum over another interval. Both operation types must be processed online: a later operation sees every earlier update.

This is the local blind exercise. It combines the two one-sided patterns from the section, but the statement does not reveal how to make both the range update and the range-sum query logarithmic.

5.2 Constraints and traps

$n, q \leq 200,000$, so literally adding to every position of a long interval is too slow. An operation sequence with 200,000 full-length updates would take quadratic time with a plain list.

The update amount can be negative. Sums may have magnitude up to $200,000 * 200,000 * 10^6 = 4 * 10^{16}$, which fits in signed 64-bit but not in `int`. Store Fenwick cells, update products, and answers in `long long` in C++.

The difference-array endpoint at $r+1$ is real even when $r=n$. Allocate each Fenwick tree with an extra usable index. Omitting that index lets an update leak past the end of the array instead of cancelling there.

5.3 Observations

Let d be the difference array. A range add is small in that representation:

```
add x on [l,r] means d[l] += x, d[r+1] -= x.
```

The current value at position i is the prefix $d[1] + \dots + d[i]$. A single Fenwick tree over d therefore solves range-add/point-query, as in problem C. For a range sum, we need the prefix of those point values:

```
P(p) = value[1] + value[2] + ... + value[p]
      = sum(i=1..p) sum(j=1..i) d[j]
      = sum(j=1..p) d[j] * (p-j+1)
      = (p+1) * sum d[j] - sum j*d[j].
```

This last line is the key. Maintain two Fenwick trees:

B1 stores $d[j]$	so $B1.prefix(p) = \sum d[j]$
B2 stores $j*d[j]$	so $B2.prefix(p) = \sum j*d[j]$

Then:

```
P(p) = (p+1) * B1.prefix(p) - B2.prefix(p).
sum(l,r) = P(r) - P(l-1).
```

5.4 Sample walk-through

After add 3 on $[2,4]$, the difference array has nonzero entries:

```
d[2] = +3, d[5] = -3.
```

For $p=4$, the two maintained prefixes are:

```
B1.prefix(4) = 3
B2.prefix(4) = 2*3 = 6
P(4) = (4+1)*3 - 6 = 9.
```

That is the sum of $[0,3,3,3]$. After adding -1 to $[1,5]$, the query $[2,3]$ is $P(3)-P(1)=5-1=4$. The final $+10$ at index three makes $[2,3]$ into $[2,12]$; the final query $[3,5]$ is $12+2-1=13$.

5.5 Algorithm

```
make two Fenwick trees B1 and B2 with indices 1..n+1

range_add(l, r, x):
    add B1 at l with x
    add B1 at r+1 with -x
    add B2 at l with l*x
    add B2 at r+1 with -(r+1)*x

prefix_sum(p):
    return (p+1) * B1.prefix(p) - B2.prefix(p)

for each operation:
    if it is type 1: range_add(l, r, x)
    otherwise: print prefix_sum(r) - prefix_sum(l-1)
```

$prefix_sum(0)$ is zero because both Fenwick prefix queries at index zero are zero. This makes the $l=1$ range-query boundary automatic.

5.6 Correctness proof

We prove that every printed query answer is correct.

Lemma 1. After all updates processed so far, `B1.prefix(p)` equals $\sum_{j=1..p} d[j]$ and `B2.prefix(p)` equals $\sum_{j=1..p} j*d[j]$ for the current difference array.

Proof. Each range update changes exactly `d[1]` by `x` and `d[r+1]` by `-x`. The algorithm applies those same changes to `B1`, and applies each change multiplied by its index to `B2`. A Fenwick prefix query is the sum of its stored point values, so both stated equalities hold after every update. $\square$

Lemma 2. `prefix_sum(p)` returned by the algorithm equals the sum of the first `p` logical array values.

Proof. The logical value at position `i` is $\sum_{j=1..i} d[j]$. Swapping the order of summation gives:

```
sum(i=1..p) value[i]
= sum(j=1..p) d[j]*(p-j+1)
= (p+1)*sum(j=1..p)d[j] - sum(j=1..p)j*d[j].
```

By Lemma 1, the two Fenwick prefixes are exactly the two sums on the last line, which is the formula used by `prefix_sum`. $\square$

Theorem. A type-2 query `[1,r]` is printed with its correct range sum.

Proof. By Lemma 2, `prefix_sum(r)` is the sum of positions `1..r`, and `prefix_sum(1-1)` is the sum of positions `1..1-1`. Their difference leaves exactly the positions `1..r`. $\square$

5.7 Complexity

Each update uses four Fenwick point additions, and each query uses four Fenwick prefix sums:

```
Time:    O(log n) per operation
Memory:  O(n)
```

5.8 C++ implementation notes

Keep the trees indexed through `n+1`. In the weighted tree, write `delta * index` with `delta` as `long long`; otherwise `int` multiplication can overflow before it reaches the tree.

Lambdas for `range_add` and `prefix_sum` keep the four coordinated updates and the one formula together, reducing endpoint-copy mistakes.

5.9 Python implementation notes

Python integers safely hold all intermediate products. Parse the full input once and maintain an index into the token list; append answers to a list and write it once. The Fenwick loops still use ordinary integer indices.

5.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<long long> bit;

    explicit Fenwick(int size) : n(size), bit(size + 1, 0) {}

    void add(int index, long long delta) {
        for (; index <= n; index += index & -index) bit[index] += delta;
    }

    long long sum(int index) const {
        long long result = 0;
        for (; index > 0; index -= index & -index) result += bit[index];
        return result;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    Fenwick coefficients(n + 1), weighted_coefficients(n + 1);

    auto add_range = [&](int left, int right, long long delta) {
        coefficients.add(left, delta);
        coefficients.add(right + 1, -delta);
        weighted_coefficients.add(left, delta * left);
        weighted_coefficients.add(right + 1, -delta * (right + 1));
    };

    auto prefix_sum = [&](int position) {
        long long coefficient_sum = coefficients.sum(position);
        long long weighted_sum = weighted_coefficients.sum(position);
        return (position + 1LL) * coefficient_sum - weighted_sum;
    };

    while (q--) {
        int type, left, right;
        cin >> type >> left >> right;
        if (type == 1) {
            long long delta;
            cin >> delta;
            add_range(left, right, delta);
        } else {
```

```

        cout << prefix_sum(right) - prefix_sum(left - 1) << '\n';
    }
}
return 0;
}

```

5.11 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, size: int) -> None:
        self.size = size
        self.bit = [0] * (size + 1)

    def add(self, index: int, delta: int) -> None:
        while index <= self.size:
            self.bit[index] += delta
            index += index & -index

    def sum(self, index: int) -> int:
        result = 0
        while index > 0:
            result += self.bit[index]
            index -= index & -index
        return result

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    coefficients = Fenwick(n + 1)
    weighted_coefficients = Fenwick(n + 1)

    def add_range(left: int, right: int, delta: int) -> None:
        coefficients.add(left, delta)
        coefficients.add(right + 1, -delta)
        weighted_coefficients.add(left, delta * left)
        weighted_coefficients.add(right + 1, -delta * (right + 1))

    def prefix_sum(position: int) -> int:
        return (position + 1) * coefficients.sum(position) - weighted_coefficients.sum(position)

    index = 2
    output: list[str] = []

```

```

for _ in range(q):
    operation = data[index]
    left = data[index + 1]
    right = data[index + 2]
    index += 3
    if operation == 1:
        delta = data[index]
        index += 1
        add_range(left, right, delta)
    else:
        output.append(str(prefix_sum(right) - prefix_sum(left - 1)))
sys.stdout.write("\n".join(output) + ("\n" if output else ""))

if __name__ == "__main__":
    main()

```

5.12 Common wrong answers

Using one tree over differences and treating its prefix as a range sum.
 Forgetting to multiply the B2 updates by their exact endpoint index.
 Using $p \cdot B1.prefix(p) - B2.prefix(p)$ instead of $(p+1) \cdot \dots$.
 Not allocating the $r+1$ endpoint when $r=n$.
 Using int for values or products after many updates.

5.13 Tests that matter

$n=1$ with positive and negative updates.
 Full-range updates where $r=n$, to exercise the cancellation endpoint.
 Queries with $l=1$, where $prefix_sum(0)$ is needed.
 Overlapping updates of opposite signs.
 200,000 updates and queries, to reject linear scans and test 64-bit totals.

6 Testing discussion

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs with a literal mutable array, then the same expected files are used against both the C++ and Python submissions. For D, include full-range updates, negative deltas, and queries whose left endpoint is one.

7 Problem-specific upsolve notes

7.1 A. Point Add Range Sum

Constraint implication and trap. With a long sequence of updates and queries, scanning $[1, r]$ per query is quadratic in the worst case. The update is an addition, not an assignment: call `add(i, x)`, never overwrite the old Fenwick contribution.

Walk-through. From $[1, 2, 3, 4]$, query $[2, 4]$ is `prefix(4)-prefix(1)=9`. After `add(3, 10)`, the same query is 19. The tree's prefix invariant proves the subtraction removes exactly positions before 1.

```
build: for i=1..n: add(i, a[i])
type 1 i x: add(i, x)
type 2 l r: print prefix(r)-prefix(l-1)
```

Every `bit[j]` stores its lowbit block sum. Prefix blocks partition $[1, r]$ and $[1, l-1]$, so their difference is exactly $[l, r]$. Build is $O(n \log n)$ with these calls and each operation is $O(\log n)$; memory is $O(n)$. In both C++ and Python, use 64-bit-capable sums. Test $l=1$, $l=r$, negative delta, and a query after several additions to one position.

7.2 B. Inversion Count

Observation. Values can be large or negative, so compress them to ordered ranks. At index i , $i - \text{prefix}(\text{rank})$ is the number of earlier values that are strictly greater; using `prefix(rank-1)` would incorrectly count equals.

For $[3, 1, 2, 5, 4]$, the contributions are $0, 1, 1, 0, 1$, total 3. The Fenwick invariant counts exactly the processed values of each rank, and compression preserves $<$, proving the accumulated contributions count every inversion once when its right endpoint arrives. Complexity is $O(n \log n)$ time and $O(n)$ memory. C++ needs `long long inv`; Python needs no special numeric type. Test sorted, reverse-sorted, all-equal, and values outside 32-bit range.

```
compress values to ranks
for i from 0 to n-1:
    answer += i - prefix(rank[a[i]])
    add(rank[a[i]], 1)
```

7.3 C. Range Add Point Query

Observation. Store `diff`, not the logical values. A range add changes only `diff[l]` and `diff[r+1]`; a point value is the prefix sum of differences. For `[10,20,30,40]`, add 5 on `[2,3]`: prefix at 3 changes from 30 to 35, but prefix at 4 returns to 40 because of `diff[4]=-5`.

The difference identity proves that every position in `[1,r]` sees `+x` once and every other position sees net zero. Each update and point query is $O(\log n)$, with $O(n)$ memory. Allocate an extra usable index for `r+1`. In C++/Python, initialize by inserting `a[i]-a[i-1]`, not `a[i]`. Test a full-range add, `r=n`, and a negative update.

8 Stress-testing workflow

For A and C, keep a slow list and mutate it literally; for B, use a double loop on arrays of length at most eight. Compare after every random operation, then run `CP_TARGET=solution python3 sections/36_fenwick_trees/check.py`. Carry forward: Fenwick trees are prefix structures, and coordinate compression or a difference transform changes what that prefix means.